

Practical Problems: Efficiency

Learning Outcome Addressed

- Recognize differences in algorithmic approaches based on computational efficiency.
- Recognize differences in basic data structures such as lists, sets, and dictionaries, based on computational efficiency.
- Interpret and write code using operators, functions, and methods on sets and dictionaries.

This assignment allows you to apply what you have learned in this module about computational efficiency. Follow the instructions below to complete problems one to seven. If you get stuck on a problem and are not making any progress, please reach out to the learning facilitator for assistance.

1. [20 pts] `average_of_max_scores`

Write the function **`average_of_max_scores`** that takes in a 2D list where each row has 2 entries: a student username and a score. The students are allowed to make multiple submissions, so there can be multiple rows with the same username.

A sample input may be

```
[["alice", 70], ["bob", 70], ["alice", 80], ["charlie", 90]]
```

The function should be the average of the maximum scores of the students. In the example above, Alice's max score is 80, Bob's is 70, and Charlie's is 90. So the returned value is the average among these scores: 80.

2. [20 pts] `merge_dicts`

Write the function **`merge_dicts(L)`** that takes in a list of dictionaries and non-destructively merges them.

For example if `L = [{"a": 1, "b": 2, "c": 3}, {"a": 4, "b": 2, "c": 3}]`, then `merge_dicts(L)` should return `{"a": {1, 4}, "b": 2, "c": 3}`

Notice that if the two dictionaries have the same key with different values, we create a set from those values. For this question, we'll assume the

dictionaries given as input have immutable values (otherwise we would not be able to add those values to a set).

3. [20 pts] intersect

Write the function **intersect(L, K)** that takes in two lists and returns a new list consisting of all the elements of K that are also in L. The result should preserve the order of the elements in K and keep duplicated values.

For example, if

L = [1, 2, 2, 3, 0]

K = [1, 1, 2, 5, 0]

then the returned list should be

[1, 1, 2, 0].

The running time of your solution should be linear time.

4. [20 pts] duplicated_ids

Write the function **duplicated_ids** that takes in a list of students, where each student is represented by a dictionary, which is guaranteed to have a key named "id" representing the unique student id number (as an int). The dictionary can contain other kinds of information like "major", "gpa", and so on.

For example, the input list could be

```
[ {"id": 1, "major": "computer science", "gpa": 4},  
  {"id": 2, "major": "mathematics", "gpa": 3.5},  
  {"id": 3, "major": "chemical engineering", "gpa": 3.7} ]
```

The list may contain multiple entries with the same value for the "id" key. The goal is to return a list of id values which are duplicated in the original list. In the example above, the function would return an empty list because there are no duplicate ids.

The running time of your solution should be linear time.

5. [20 pts] complete_friendship

Write the function **complete_friendship(d)** that takes in a dictionary `d` which represents friendship relationships. The keys are names where `d[name]` maps to a set of other names that the person is friends with.

For example, `d` could be equal to

```
{"alice": {"bob", "charlie"},  
"eve": {"alice"}}
```

indicating that alice is friends with bob and charlie, and eve is friends with alice. We assume that friendship is a symmetric relationship. That is, if `x` is friends with `y`, then `y` is friends with `x`. So given a dictionary as above, we want to modify it to reflect this relationship.

For the input dictionary above, the result dictionary would become

```
{"alice": {"eve", "bob", "charlie"},  
"eve": {"alice"},  
"bob": {"alice"},  
"charlie": {"alice"}}
```

This function should be destructive – in other words, it should directly modify the inputted dictionary and have no return statements.

6. [25 pts] max_clique

Write the function **max_clique(G)** that takes in a dictionary that represents friendship relationships among a set of people. The keys of `G` represent names, which map to an inner dictionary that maps a name to a boolean. If `x` and `y` denote two names, then `G[x][y]` is `True` if `x` and `y` are friends, and `False` otherwise. For example, suppose there are 3 people, Alice, Bob and Charlie, where Alice is friends with both Bob and Charlie, but Bob and Charlie are not friends. Then, `G` would look like:

```
{"Alice": {"Alice": True, "Bob": True, "Charlie": True},  
"Bob": {"Alice": True, "Bob": True, "Charlie": False},  
"Charlie": {"Alice": True, "Bob": False, "Charlie": True}}
```

Note that we assume a person is friends with themselves.

We define a clique as a set of names/people who are all friends with each other. Your function should return the largest clique given such a dictionary G. In the example above, the function would return {"Alice", "Bob"}.

There are a couple of important notes. First, the efficiency of your solution is not important. In fact, from a computer science perspective, there is strong evidence that the best possible algorithm for solving this problem has exponential time complexity.

Second, to solve this problem, you will probably need a way to generate all subsets of a given set. We have provided a function that does this for you (`all_subsets`). Given a set S, it returns a list of subsets of S. It is a recursive function. We encourage you to study it and try to understand how it works. But that being said, understanding exactly how the function works is not required to solve this problem.

7. [25 pts] `index_range`

Write the function **`index_range`** that takes in a sorted list L of integers and a target integer. We want to return a list of two indices [i, j] where i is the first index such that $L[i] = \text{target}$ and j is the last index such that $L[j] = \text{target}$. If target is not in L, then return [-1, -1].

Your solution must run in logarithmic time. (Note that it is possible the whole list consists of a single number.)